# HW5

Name: Minhao Zou
Github Username: zmh0917
USC ID: 6793831877

## 1. Decision Trees as Interpretable Models

Import packages

In [1]:
```
from pathlib import Path
from importlib.metadata import version
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

from sklearn.tree import DecisionTreeClassifier, plot_tree, _tree
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold, GridSearchCV, cross_val_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.decomposition import PCA
from xgboost import XGBRegressor
```

### (a) Obtain Data

Get the Accute Inamations Data Set

In [2]:
```
columns = ["Temperature","Nausea","Lumbar pain", "Urine pushing", "Micturition p
df = pd.read_csv(
    Path("data") / "diagnosis.data",
    sep="\t",
    header=None,
    encoding="utf-16",
    decimal=",",
    names=columns
)

for column in columns[1:]:
    df[column] = (df[column].str.strip().map({"no": 0, "yes": 1}))
X = df[columns[:6]].copy()
y = 2*df["Bladder inflammation"]+df["Nephritis"]

print(df.shape)
print(df.isna().sum().sum())
```

```
print(y.value_counts().sort_index())

display(df.head())
```

```
(120, 8)
0
0    30
1    31
2    40
3    19
Name: count, dtype: int64
```

| | Temperature | Nausea | Lumbar pain | Urine pushing | Micturition pains | Urethra burning | Bladder inflammation | Ne |
|---|---|---|---|---|---|---|---|---|
| **0** | 35.5 | 0 | 1 | 0 | 0 | 0 | 0 | |
| **1** | 35.9 | 0 | 0 | 1 | 1 | 1 | 1 | |
| **2** | 35.9 | 0 | 1 | 0 | 0 | 0 | 0 | |
| **3** | 36.0 | 0 | 0 | 1 | 1 | 1 | 1 | |
| **4** | 36.0 | 0 | 1 | 0 | 0 | 0 | 0 | |

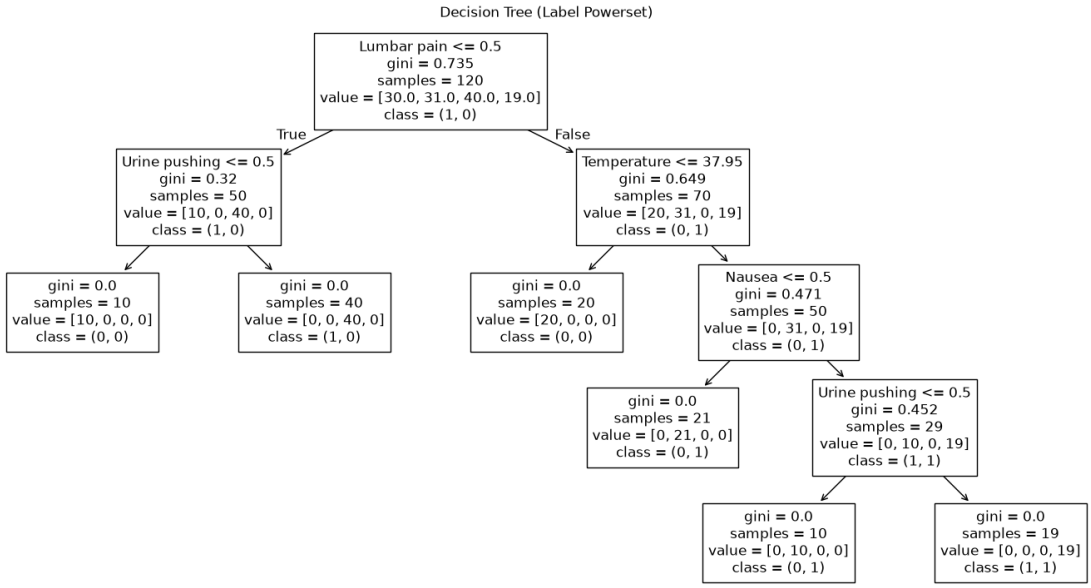
## (b) Build a decision tree

In [3]:
```
tree_model = DecisionTreeClassifier(random_state=42)
tree_model.fit(X, y)
y_pred = tree_model.predict(X)

print("train accuracy:", accuracy_score(y, y_pred))
print("depth:", tree_model.get_depth())
print("leaves num:", tree_model.get_n_leaves())

labels = {
    0: "(0, 0)",
    1: "(0, 1)",
    2: "(1, 0)",
    3: "(1, 1)"
}
class_names = [labels[k] for k in tree_model.classes_]
plt.figure(figsize=(18, 9))
plot_tree(
    tree_model,
    feature_names=list(X.columns),
    class_names=class_names,
)
plt.title("Decision Tree (Label Powerset)")
plt.show()
```

```
train accuracy: 1.0
depth: 4
leaves num: 6
```




## (c) Convert the decision rules

In [ ]:
```python
def print_rules(model, names):
    tree = model.tree_
    def visit(node, path):
        if tree.feature[node] == _tree.TREE_UNDEFINED:
            values = tree.value[node][0]
            index = np.argmax(values)
            label = int(model.classes_[index])
            bladder = label // 2
            nephritis = label % 2
            rule = " AND ".join(path) if path else "TRUE"
            print(
                f"IF {rule} THEN "
                f"Bladder inflammation={bladder}, Nephritis={nephritis}"
            )
            return

        feature = names[tree.feature[node]]
        threshold = tree.threshold[node]
        visit(tree.children_left[node], path+[f"{feature} <= {threshold:.2f}"])
        visit(tree.children_right[node],path+[f"{feature} > {threshold:.2f}"])
    visit(0, [])

print_rules(tree_model, list(X.columns))
```

```
IF Lumbar pain <= 0.50 AND Urine pushing <= 0.50 THEN Bladder inflammation=0, Nep
hritis=0
IF Lumbar pain <= 0.50 AND Urine pushing > 0.50 THEN Bladder inflammation=1, Neph
ritis=0
IF Lumbar pain > 0.50 AND Temperature <= 37.95 THEN Bladder inflammation=0, Nephr
itis=0
IF Lumbar pain > 0.50 AND Temperature > 37.95 AND Nausea <= 0.50 THEN Bladder inf
lammation=0, Nephritis=1
IF Lumbar pain > 0.50 AND Temperature > 37.95 AND Nausea > 0.50 AND Urine pushing
<= 0.50 THEN Bladder inflammation=0, Nephritis=1
IF Lumbar pain > 0.50 AND Temperature > 37.95 AND Nausea > 0.50 AND Urine pushing
> 0.50 THEN Bladder inflammation=1, Nephritis=1
```

## (d) Use cost-complexity pruning to find a minimal decision tree and a set of decision rules with high interpretability.

In [9]:
```
path = tree_model.cost_complexity_pruning_path(X, y)
alphas = np.unique(path.ccp_alphas)

rows = []
for alpha in alphas:
    model = DecisionTreeClassifier(
        random_state=42,
        ccp_alpha=alpha
    )
    model.fit(X, y)
    pred = model.predict(X)
    rows.append({
        "Alpha": alpha,
        "Depth": model.get_depth(),
        "Leaves": model.get_n_leaves(),
        "Acc": accuracy_score(y, pred)
    })

table = pd.DataFrame(rows)
display(table.round(6))
```

| | Alpha | Depth | Leaves | Acc |
|---|---|---|---|---|
| **0** | 0.000000 | 4 | 6 | 1.000000 |
| **1** | 0.098167 | 2 | 4 | 0.841667 |
| **2** | 0.133333 | 2 | 3 | 0.758333 |
| **3** | 0.182000 | 1 | 2 | 0.591667 |
| **4** | 0.222917 | 0 | 1 | 0.333333 |

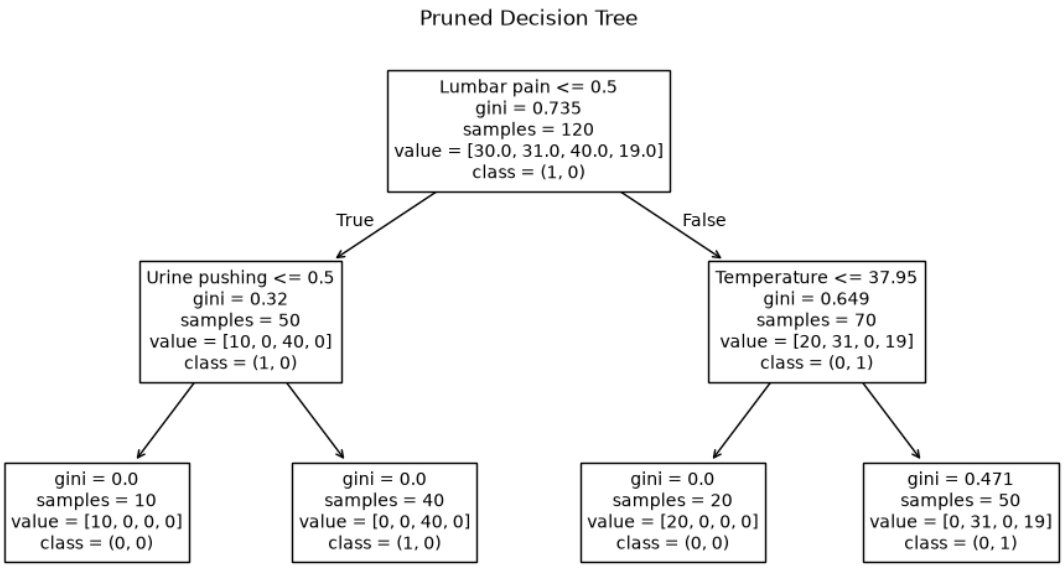
In [8]:
```
alpha = alphas[1]
pruned = DecisionTreeClassifier(
    random_state=42,
    ccp_alpha=alpha
)
pruned.fit(X, y)
pred = pruned.predict(X)

print(f"alpha: {alpha:.6f}")
print("depth:", pruned.get_depth())
print("leaves num:", pruned.get_n_leaves())
print(f"train acc: {accuracy_score(y, pred):.2%}")

plt.figure(figsize=(12, 6))
plot_tree(
    pruned,
    feature_names=list(X.columns),
    class_names=class_names,
)
plt.title("Pruned Decision Tree")
plt.show()
```

```
print_rules(pruned, list(X.columns))
```

```
alpha: 0.098167
depth: 2
leaves num: 4
train acc: 84.17%
```




```
IF Lumbar pain <= 0.50 AND Urine pushing <= 0.50 THEN Bladder inflammation=0, Nep
hritis=0
IF Lumbar pain <= 0.50 AND Urine pushing > 0.50 THEN Bladder inflammation=1, Neph
ritis=0
IF Lumbar pain > 0.50 AND Temperature <= 37.95 THEN Bladder inflammation=0, Nephr
itis=0
IF Lumbar pain > 0.50 AND Temperature > 37.95 THEN Bladder inflammation=0, Nephri
tis=1
```

choose the second alpha value in the table, approximately 0.098167

After pruning, the tree depth decreases from 4 to 2, and the number of leaves decreases from 6 to 4. The training accuracy drops from 100% to 84.17%

now have only four rules, each with two conditions

# 2. The LASSO and Boosting for Regression

## (a) Obtain Data

In [10]:
```
names = []
with open(Path("data")/"communities.names", encoding="utf-8") as f:
    for line in f:
        if line.strip().startswith("@attribute"):
            names.append(line.split()[1])

crime = pd.read_csv(
    Path("data")/"communities.data",
    header=None,
    names=names,
    na_values="?"
```

```
)

train = crime.iloc[:1495].copy()
test = crime.iloc[1495:].copy()
print(crime.shape)
print(train.shape)
print(test.shape)
display(crime.head())
```

```
(1994, 128)
(1495, 128)
(499, 128)
```

| | state | county | community | communityname | fold | population | householdsize |
|---|---|---|---|---|---|---|---|
| **0** | 8 | NaN | NaN | Lakewoodcity | 1 | 0.19 | 0.33 |
| **1** | 53 | NaN | NaN | Tukwilacity | 1 | 0.00 | 0.16 |
| **2** | 24 | NaN | NaN | Aberdeentown | 1 | 0.00 | 0.42 |
| **3** | 34 | 5.0 | 81440.0 | Willingborotownship | 1 | 0.04 | 0.77 |
| **4** | 42 | 95.0 | 6096.0 | Bethlehemtownship | 1 | 0.01 | 0.55 |

5 rows × 128 columns

## (b) Missing values

In [ ]:
```
unused = ["state", "county", "community", "communityname", "fold"]
target = "ViolentCrimesPerPop"

X_train = train.drop(columns=unused+[target])
X_test = test.drop(columns=unused+[target])
y_train = train[target].copy()
y_test = test[target].copy()

print(X_train.shape)
print(X_test.shape)
print("missing val in training:", X_train.isna().sum().sum())
print("missing val in test:", X_test.isna().sum().sum())
```

```
(1495, 122)
(499, 122)
Missing val in training: 27633
Missing val in test: 9218
```

In [ ]:
```
imputer = SimpleImputer(strategy="median")
train_fill = pd.DataFrame(
    imputer.fit_transform(X_train),
    columns=X_train.columns,
    index=X_train.index
)
test_fill = pd.DataFrame(
    imputer.transform(X_test),
    columns=X_test.columns,
    index=X_test.index
)
```

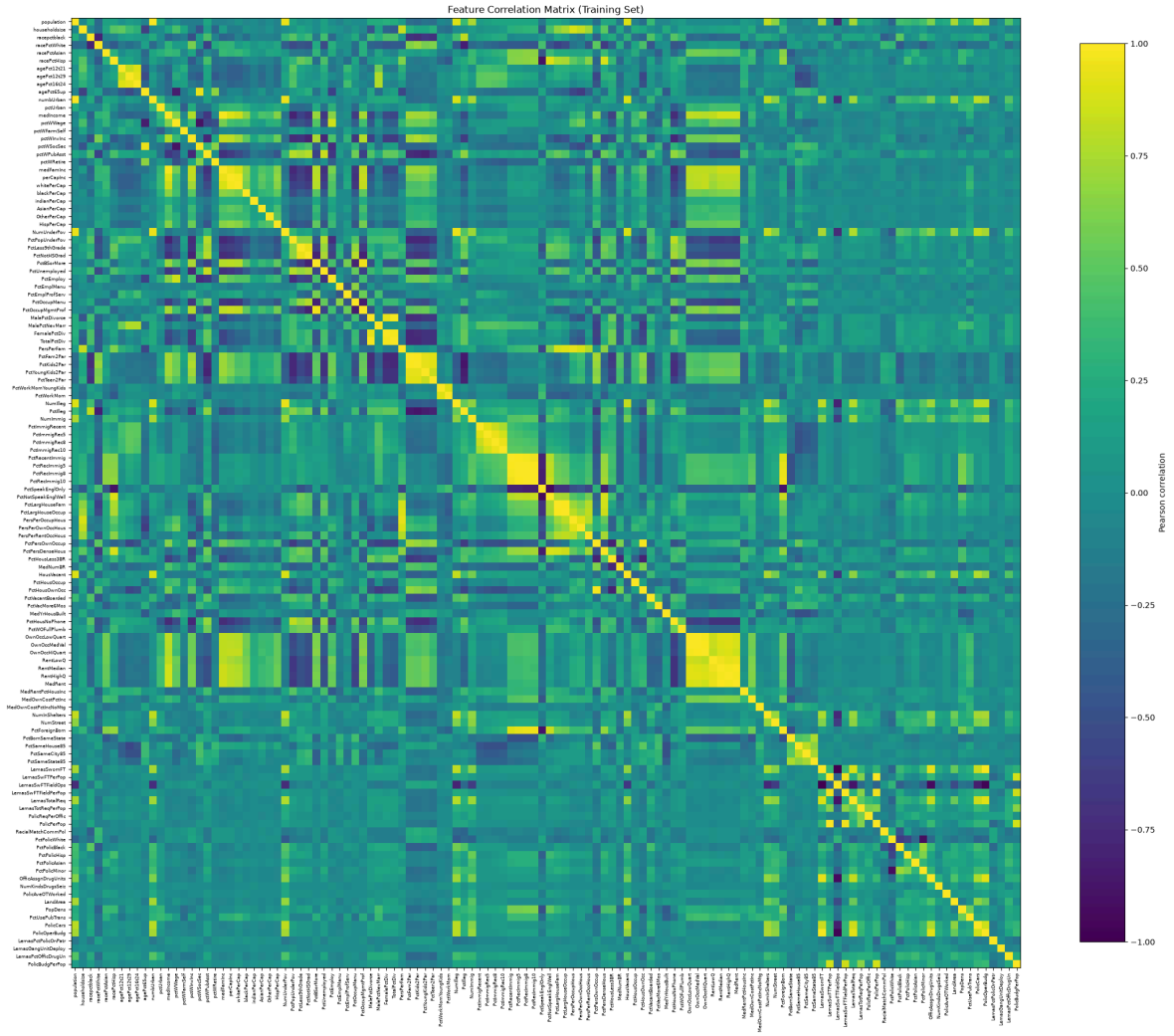
```
print("train missing:", train_fill.isna().sum().sum())
print("test missing:", test_fill.isna().sum().sum())
```

```
train: 0
test: 0
```

## (c) Plot a correlation matrix

In [13]:

```
corr = train_fill.corr()
plt.figure(figsize=(22, 20))
image = plt.imshow(
    corr,
    vmin=-1,
    vmax=1,
    interpolation="nearest"
)
plt.colorbar(image, label="Pearson correlation", shrink=0.8)
plt.xticks(
    range(len(corr)),
    corr.columns,
    rotation=90,
    fontsize=6
)
plt.yticks(
    range(len(corr)),
    corr.index,
    fontsize=6
)
plt.title("Feature Correlation Matrix (Training Set)")
plt.tight_layout()
plt.show()
# plotting assist by GPT
```




## (d) Calculate the Coefficient of Variation CV

In [16]:

```
mean = train_fill.mean()
std = train_fill.std(ddof=1)
cv = std / mean.replace(0, np.nan)

cv_table = pd.DataFrame({
    "Mean": mean,
    "Std": std,
    "CV": cv
}).sort_values("CV", ascending=False)

with pd.option_context("display.max_rows", None):
    display(cv_table.round(6))
```

| | Mean | Std | CV |
|---|---|---|---|
| **PctPolicAsian** | 0.016254 | 0.089838 | 5.527059 |
| **NumStreet** | 0.022983 | 0.098665 | 4.292923 |
| **NumInShelters** | 0.029599 | 0.102736 | 3.470952 |
| **NumIlleg** | 0.037204 | 0.113806 | 3.058964 |
| **NumImmig** | 0.030595 | 0.089541 | 2.926635 |
| **LemasPctOfficDrugUn** | 0.093304 | 0.238201 | 2.552946 |
| **NumUnderPov** | 0.056849 | 0.133167 | 2.342443 |
| **LemasSwornFT** | 0.028749 | 0.064629 | 2.248042 |
| **population** | 0.058167 | 0.130359 | 2.241105 |
| **numbUrban** | 0.064649 | 0.131784 | 2.038461 |
| **HousVacant** | 0.076475 | 0.150538 | 1.968467 |
| **PolicOperBudg** | 0.038074 | 0.062650 | 1.645508 |
| **LandArea** | 0.064107 | 0.105482 | 1.645408 |
| **racePctHisp** | 0.144963 | 0.233694 | 1.612091 |
| **PctNotSpeakEnglWell** | 0.151880 | 0.221231 | 1.456618 |
| **LemasTotalReq** | 0.049706 | 0.072167 | 1.451887 |
| **racepctblack** | 0.179839 | 0.256970 | 1.428885 |
| **PctUsePubTrans** | 0.165197 | 0.230799 | 1.397110 |
| **racePctAsian** | 0.153712 | 0.208910 | 1.359100 |
| **PctPolicHisp** | 0.063786 | 0.086359 | 1.353882 |
| **PctRecentImmig** | 0.185077 | 0.238432 | 1.288287 |
| **PctRecImmig5** | 0.184943 | 0.237871 | 1.286184 |
| **PctRecImmig10** | 0.185318 | 0.235442 | 1.270476 |
| **PctRecImmig8** | 0.187893 | 0.237793 | 1.265579 |
| **OfficAssgnDrugUnits** | 0.045933 | 0.053951 | 1.174565 |
| **PctPersDenseHous** | 0.187398 | 0.212308 | 1.132926 |
| **PolicCars** | 0.085023 | 0.094617 | 1.112831 |
| **PctForeignBorn** | 0.218281 | 0.232661 | 1.065880 |
| **PctVacantBoarded** | 0.203318 | 0.214465 | 1.054827 |
| **PctHousNoPhone** | 0.265485 | 0.244980 | 0.922765 |
| **PctIlleg** | 0.249793 | 0.230196 | 0.921550 |
| **OwnOccMedVal** | 0.262970 | 0.233888 | 0.889410 |
| **OwnOccHiQuart** | 0.268722 | 0.237438 | 0.883583 |
| **PopDens** | 0.233759 | 0.202085 | 0.864499 |

| | Mean | Std | CV |
|---|---|---|---|
| **OwnOccLowQuart** | 0.264100 | 0.227012 | 0.859566 |
| **PctWOFullPlumb** | 0.247552 | 0.208737 | 0.843206 |
| **indianPerCap** | 0.202555 | 0.165765 | 0.818368 |
| **MedNumBR** | 0.313043 | 0.254801 | 0.813947 |
| **PctPolicBlack** | 0.136776 | 0.106123 | 0.775887 |
| **PctLargHouseOccup** | 0.252261 | 0.192525 | 0.763198 |
| **PctPopUnderPov** | 0.307880 | 0.232286 | 0.754470 |
| **PctLargHouseFam** | 0.268555 | 0.198379 | 0.738692 |
| **pctWPubAsst** | 0.323445 | 0.228419 | 0.706206 |
| **pctWFarmSelf** | 0.290589 | 0.203964 | 0.701898 |
| **PctImmigRecent** | 0.324943 | 0.221444 | 0.681486 |
| **PctLess9thGrade** | 0.320147 | 0.217857 | 0.680490 |
| **OtherPerCap** | 0.281793 | 0.189725 | 0.673279 |
| **pctUrban** | 0.693318 | 0.446024 | 0.643318 |
| **RentLowQ** | 0.345773 | 0.221177 | 0.639662 |
| **AsianPerCap** | 0.326107 | 0.200512 | 0.614866 |
| **blackPerCap** | 0.291940 | 0.174971 | 0.599341 |
| **RentHighQ** | 0.421465 | 0.250375 | 0.594059 |
| **PctBSorMore** | 0.362040 | 0.214574 | 0.592680 |
| **medIncome** | 0.359217 | 0.212649 | 0.591978 |
| **PctImmigRec5** | 0.362896 | 0.214397 | 0.590793 |
| **RentMedian** | 0.371973 | 0.211570 | 0.568778 |
| **MedRent** | 0.383311 | 0.215974 | 0.563443 |
| **PctUnemployed** | 0.366602 | 0.206186 | 0.562425 |
| **perCapInc** | 0.350508 | 0.195612 | 0.558079 |
| **medFamInc** | 0.374883 | 0.202688 | 0.540670 |
| **PctNotHSGrad** | 0.387619 | 0.206893 | 0.533753 |
| **whitePerCap** | 0.368281 | 0.191443 | 0.519828 |
| **PctOccupManu** | 0.396094 | 0.204535 | 0.516379 |
| **PctEmplManu** | 0.401070 | 0.205961 | 0.513530 |
| **PctImmigRec8** | 0.401639 | 0.202857 | 0.505072 |
| **agePct16t24** | 0.335425 | 0.167816 | 0.500310 |
| **HispPerCap** | 0.387057 | 0.187362 | 0.484068 |
| **PersPerRentOccHous** | 0.401686 | 0.190260 | 0.473655 |

| | Mean | Std | CV |
|---|---|---|---|
| **MedYrHousBuilt** | 0.489324 | 0.230355 | 0.470762 |
| **MedOwnCostPctIncNoMtg** | 0.405552 | 0.190693 | 0.470207 |
| **PctImmigRec10** | 0.429940 | 0.196804 | 0.457748 |
| **PctPolicMinor** | 0.210074 | 0.095380 | 0.454029 |
| **PctVacMore6Mos** | 0.430328 | 0.188519 | 0.438082 |
| **PctOccupMgmtProf** | 0.440722 | 0.190687 | 0.432670 |
| **MedOwnCostPctInc** | 0.448308 | 0.186273 | 0.415502 |
| **agePct65up** | 0.428569 | 0.176903 | 0.412776 |
| **PolicBudgPerPop** | 0.156515 | 0.063960 | 0.408650 |
| **MalePctNevMarr** | 0.435666 | 0.176036 | 0.404062 |
| **PctEmplProfServ** | 0.441512 | 0.177862 | 0.402849 |
| **MalePctDivorce** | 0.462622 | 0.184756 | 0.399366 |
| **LemasTotReqPerPop** | 0.177030 | 0.068452 | 0.386669 |
| **TotalPctDiv** | 0.495478 | 0.185262 | 0.373905 |
| **agePct12t21** | 0.422649 | 0.155993 | 0.369083 |
| **PersPerOccupHous** | 0.459860 | 0.168885 | 0.367253 |
| **pctWInvInc** | 0.495786 | 0.181128 | 0.365335 |
| **pctWSocSec** | 0.475619 | 0.171563 | 0.360715 |
| **FemalePctDiv** | 0.488308 | 0.175981 | 0.360390 |
| **householdsize** | 0.461732 | 0.164284 | 0.355800 |
| **PctEmploy** | 0.495545 | 0.174160 | 0.351450 |
| **PctPersOwnOccup** | 0.561425 | 0.196055 | 0.349210 |
| **pctWRetire** | 0.478441 | 0.165264 | 0.345421 |
| **PolicPerPop** | 0.185572 | 0.063886 | 0.344267 |
| **LemasSwFTPerPop** | 0.185572 | 0.063886 | 0.344267 |
| **PctHousLess3BR** | 0.497940 | 0.171069 | 0.343553 |
| **MedRentPctHousInc** | 0.489819 | 0.168040 | 0.343065 |
| **PctWorkMomYoungKids** | 0.499492 | 0.170167 | 0.340681 |
| **PolicAveOTWorked** | 0.286970 | 0.096500 | 0.336271 |
| **PctBornSameState** | 0.611311 | 0.205550 | 0.336245 |
| **PctHousOwnOcc** | 0.547017 | 0.183868 | 0.336129 |
| **PctWorkMom** | 0.524047 | 0.176089 | 0.336018 |
| **LemasGangUnitDeploy** | 0.492308 | 0.164974 | 0.335104 |
| **PctKids2Par** | 0.619538 | 0.207468 | 0.334876 |

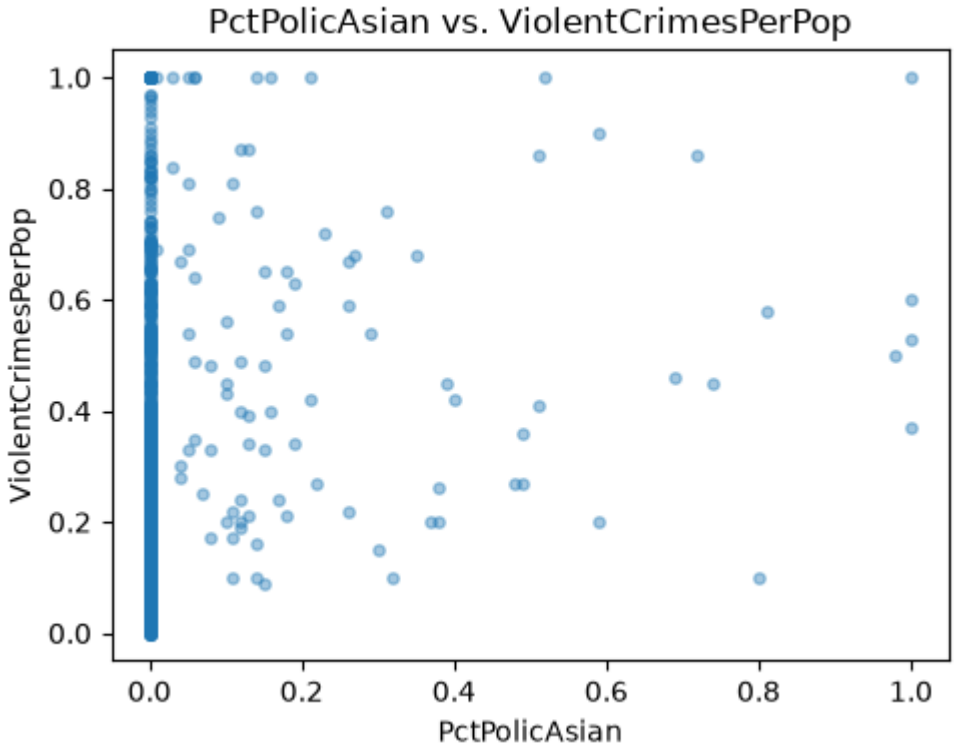
| | Mean | Std | CV |
|---|---|---|---|
| **PctSameHouse85** | 0.537365 | 0.179759 | 0.334519 |
| **PctYoungKids2Par** | 0.661157 | 0.220716 | 0.333833 |
| **PctFam2Par** | 0.609398 | 0.203048 | 0.333195 |
| **racePctWhite** | 0.752829 | 0.248594 | 0.330213 |
| **pctWWage** | 0.553498 | 0.181887 | 0.328614 |
| **PctTeen2Par** | 0.583264 | 0.191194 | 0.327801 |
| **PersPerOwnOccHous** | 0.493070 | 0.157395 | 0.319214 |
| **PctSameCity85** | 0.629572 | 0.200681 | 0.318758 |
| **PersPerFam** | 0.486682 | 0.154461 | 0.317375 |
| **PctSameState85** | 0.655906 | 0.195745 | 0.298434 |
| **agePct12t29** | 0.492087 | 0.143352 | 0.291315 |
| **PctSpeakEnglOnly** | 0.784047 | 0.227373 | 0.289999 |
| **LemasSwFTFieldPerPop** | 0.215639 | 0.061680 | 0.286033 |
| **PctHousOccup** | 0.721030 | 0.193367 | 0.268182 |
| **PolicReqPerOffic** | 0.307124 | 0.079514 | 0.258900 |
| **NumKindsDrugsSeiz** | 0.567130 | 0.079977 | 0.141021 |
| **RacialMatchCommPol** | 0.722375 | 0.093657 | 0.129652 |
| **PctPolicWhite** | 0.762856 | 0.090572 | 0.118727 |
| **LemasPctPolicOnPatr** | 0.752495 | 0.084304 | 0.112032 |
| **LemasSwFTFieldOps** | 0.961880 | 0.061958 | 0.064414 |

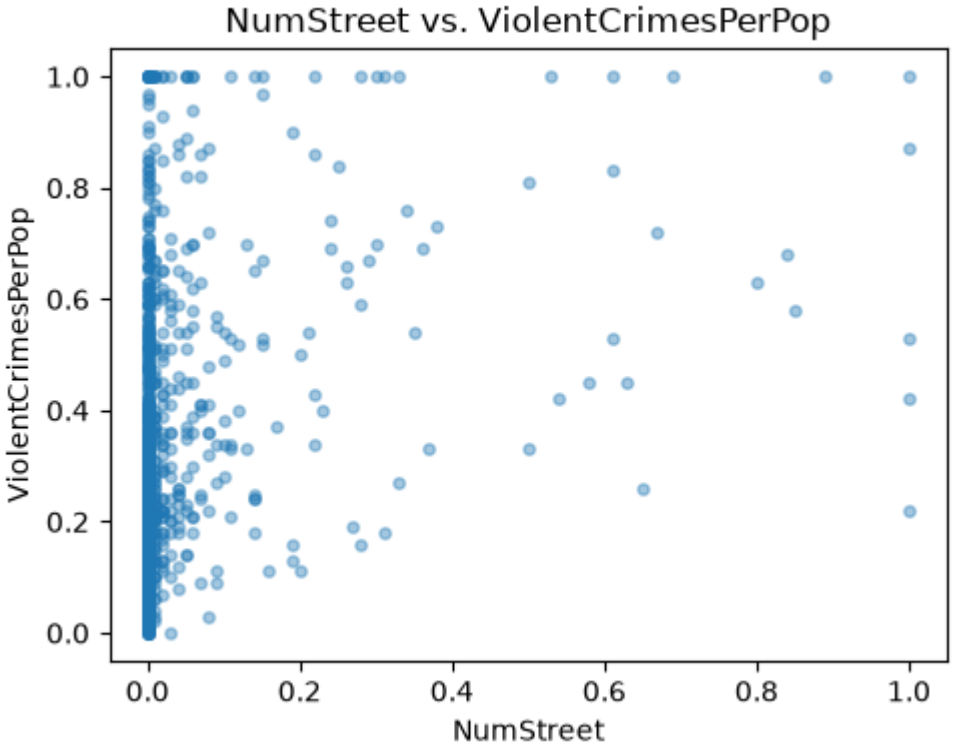
## (e) Scatter plots and box plots for highest CV features

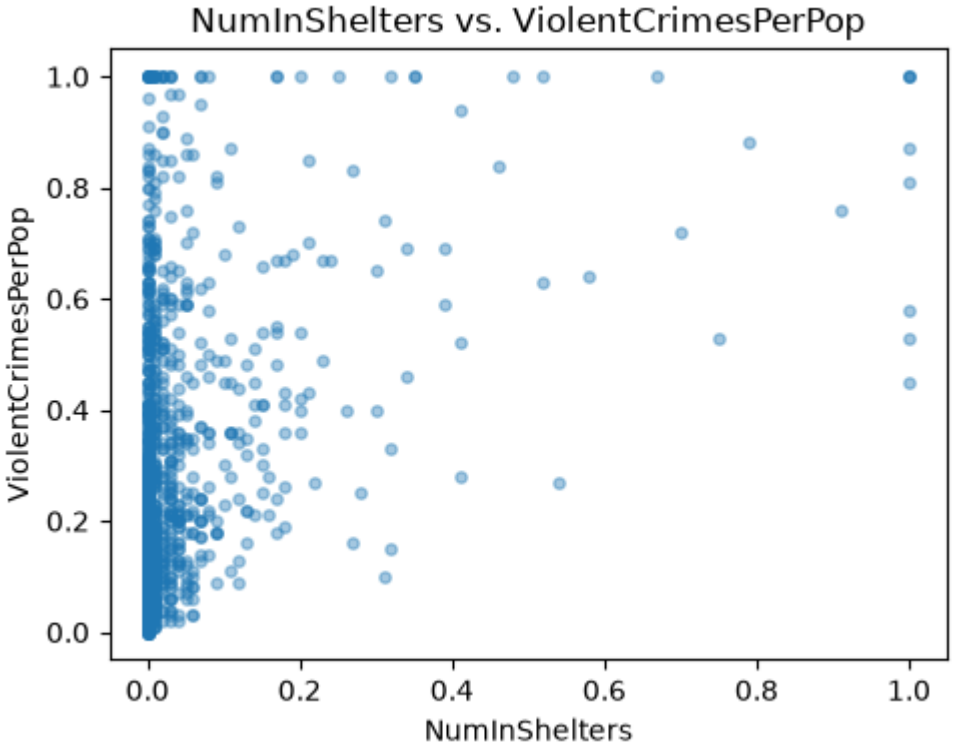
In [17]:
```
k = int(np.sqrt(128))
top = cv_table["CV"].dropna().head(k).index.tolist()
display(cv_table.loc[top].round(6))
```

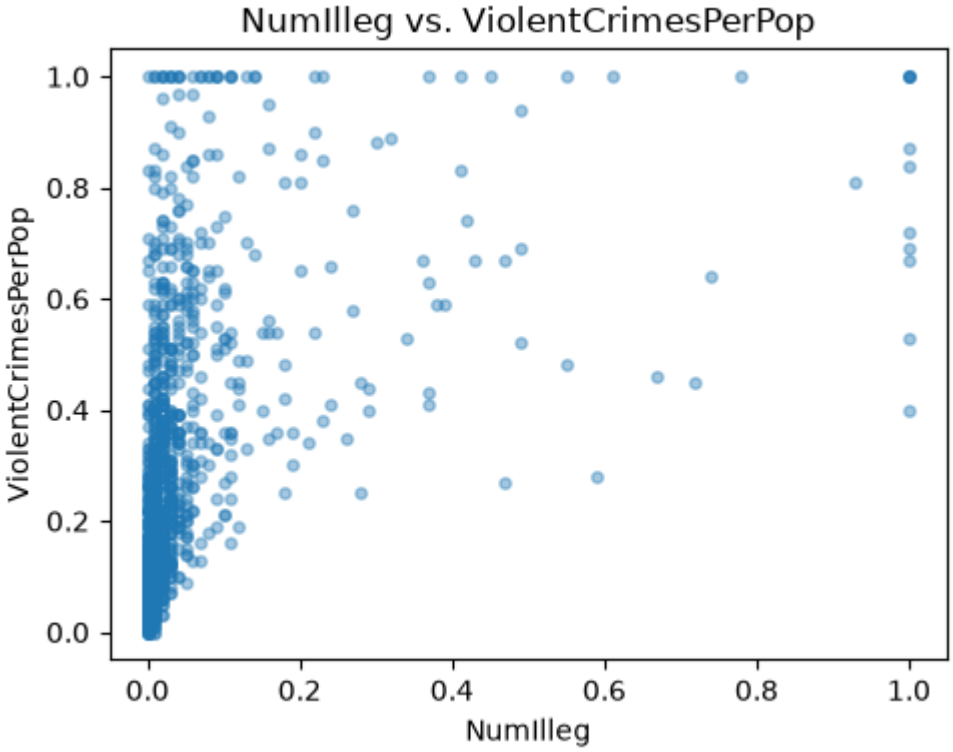
| | Mean | Std | CV |
|---|---|---|---|
| **PctPolicAsian** | 0.016254 | 0.089838 | 5.527059 |
| **NumStreet** | 0.022983 | 0.098665 | 4.292923 |
| **NumInShelters** | 0.029599 | 0.102736 | 3.470952 |
| **NumIlleg** | 0.037204 | 0.113806 | 3.058964 |
| **NumImmig** | 0.030595 | 0.089541 | 2.926635 |
| **LemasPctOfficDrugUn** | 0.093304 | 0.238201 | 2.552946 |
| **NumUnderPov** | 0.056849 | 0.133167 | 2.342443 |
| **LemasSwornFT** | 0.028749 | 0.064629 | 2.248042 |
| **population** | 0.058167 | 0.130359 | 2.241105 |
| **numbUrban** | 0.064649 | 0.131784 | 2.038461 |
| **HousVacant** | 0.076475 | 0.150538 | 1.968467 |

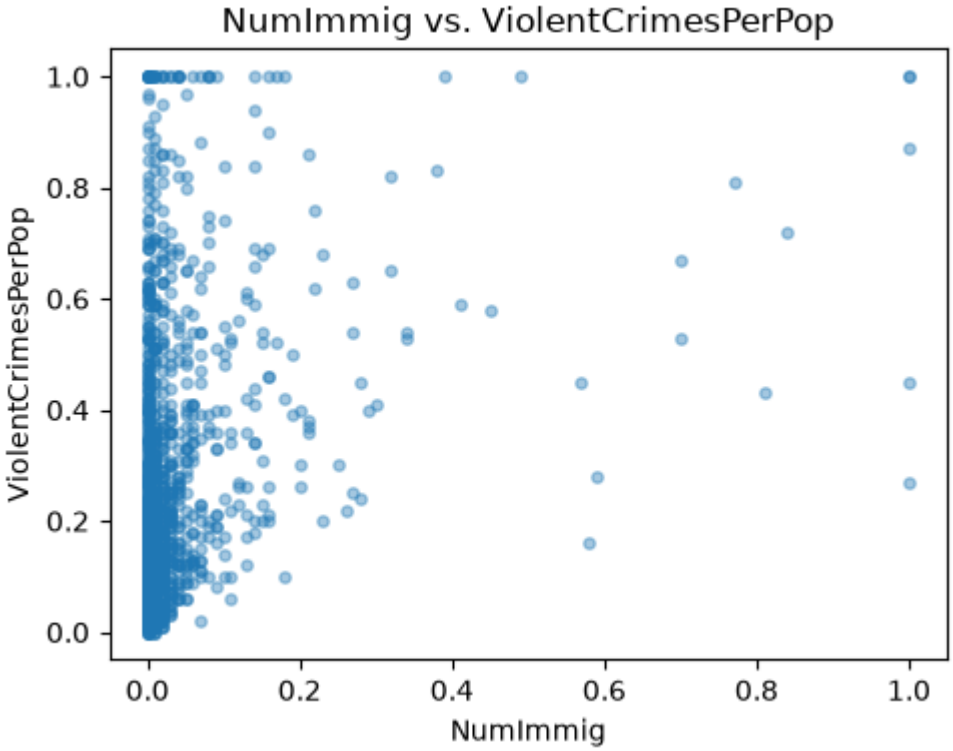
In [18]:
```
for name in top:
    plt.figure(figsize=(5, 4))
    plt.scatter(
        train_fill[name],
        y_train,
        s=15,
        alpha=0.4
    )
    plt.xlabel(name)
    plt.ylabel(target)
    plt.title(f"{name} vs. {target}")
    plt.tight_layout()
    plt.show()
```

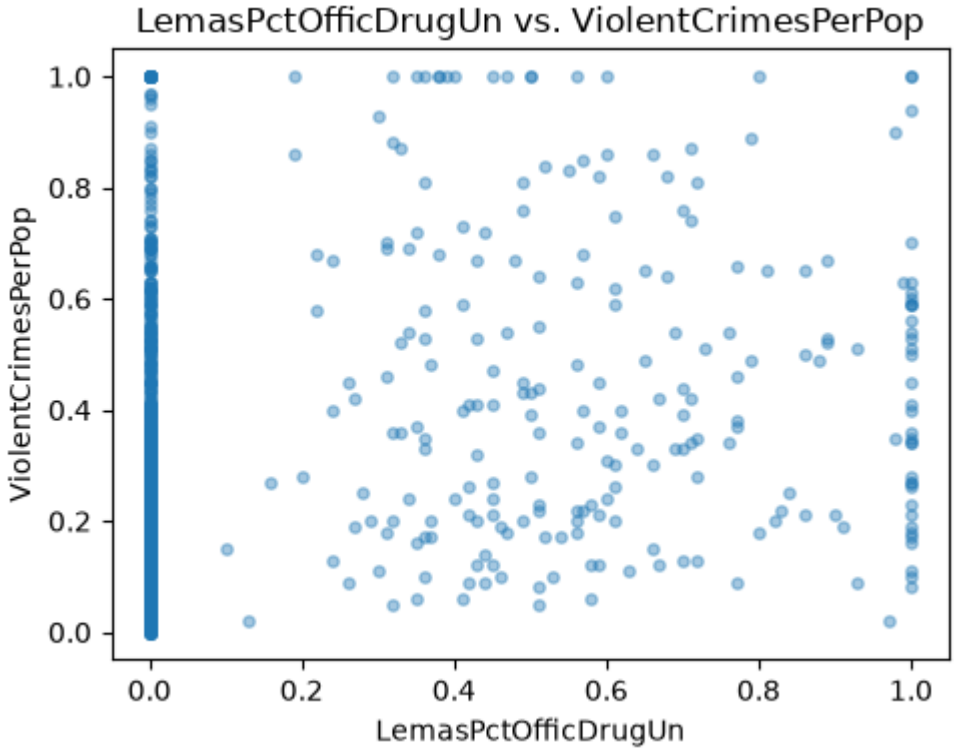


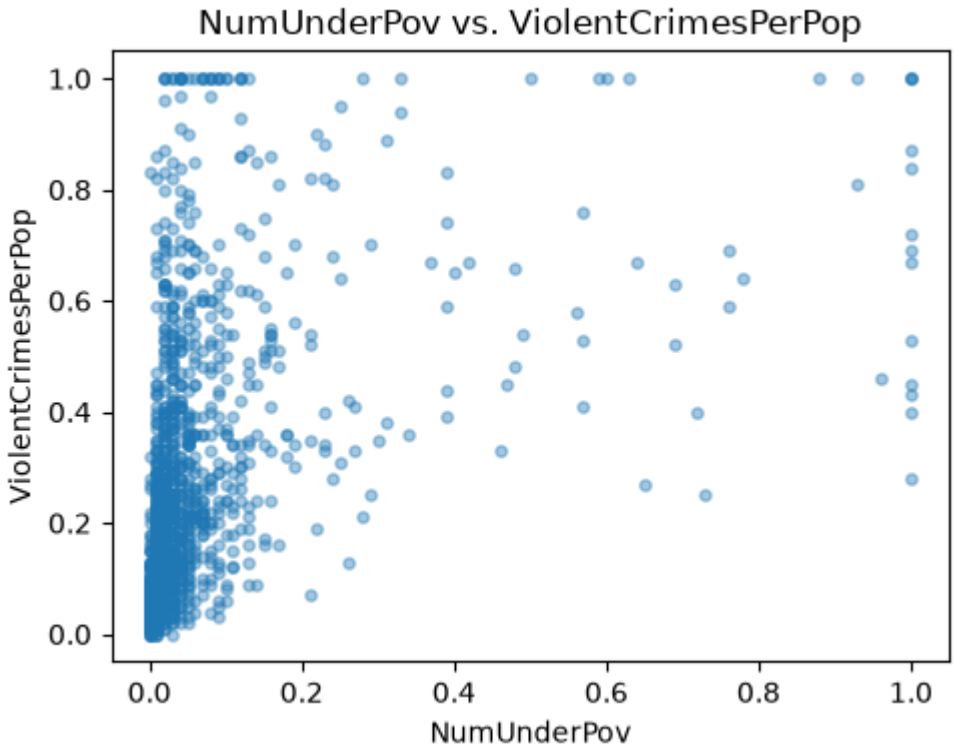

NumStreet vs. ViolentCrimesPerPop
ViolentCrimesPerPop
NumStreet
0.0
0.2
0.4
0.6
0.8
1.0

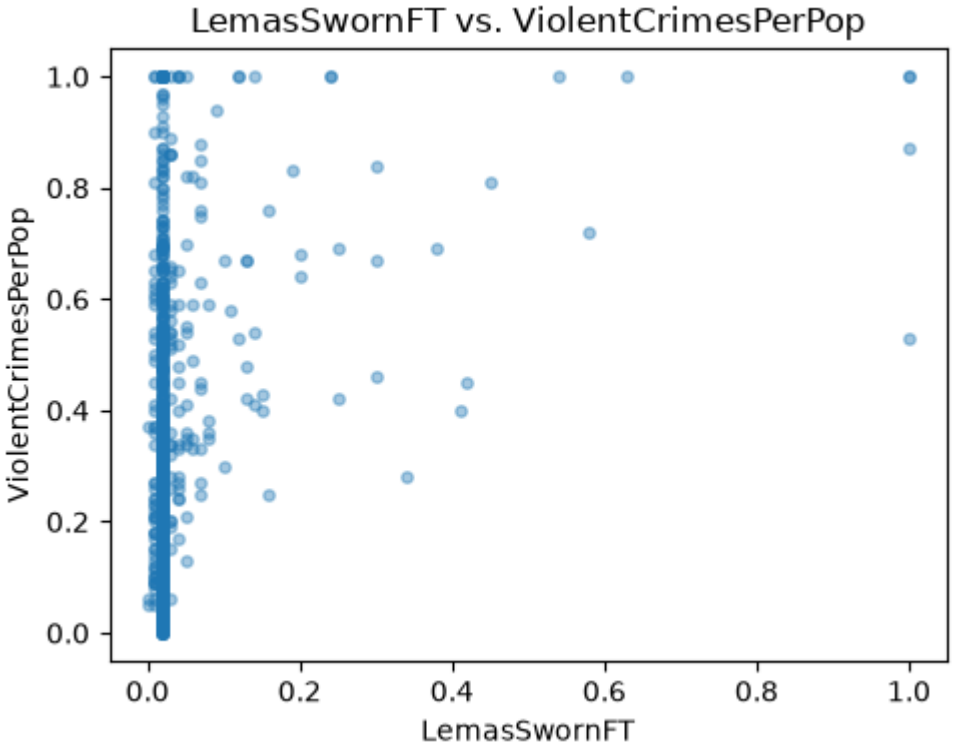

NumInShelters vs. ViolentCrimesPerPop
ViolentCrimesPerPop
NumInShelters
0.0
0.2
0.4
0.6
0.8
1.0

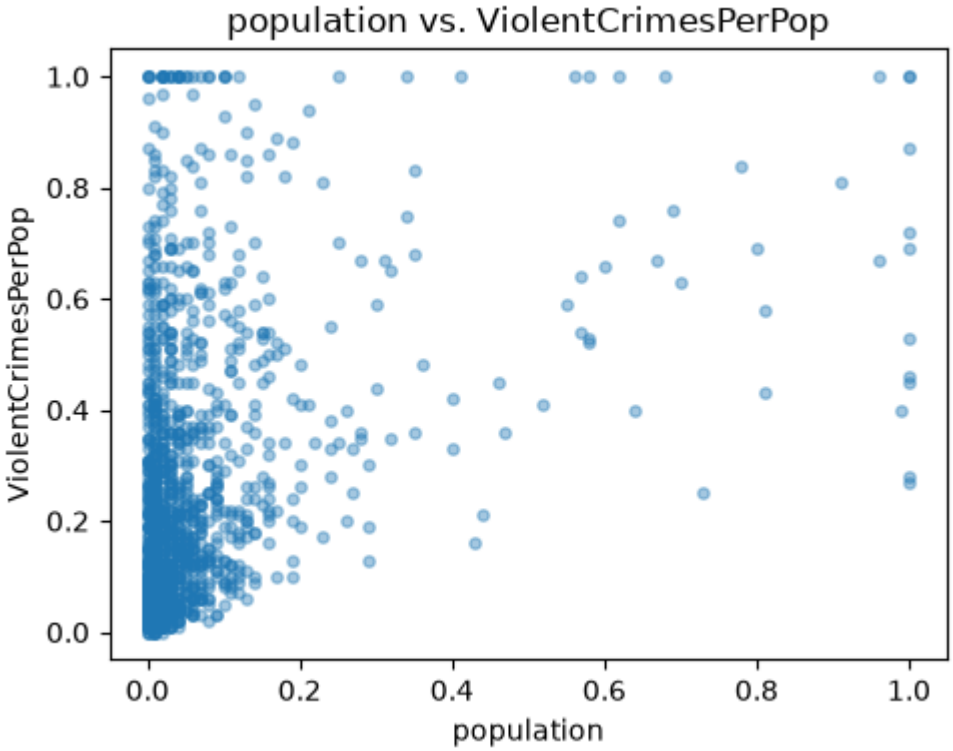

NumIlleg vs. ViolentCrimesPerPop
ViolentCrimesPerPop
NumIlleg
0.0
0.2
0.4
0.6
0.8
1.0

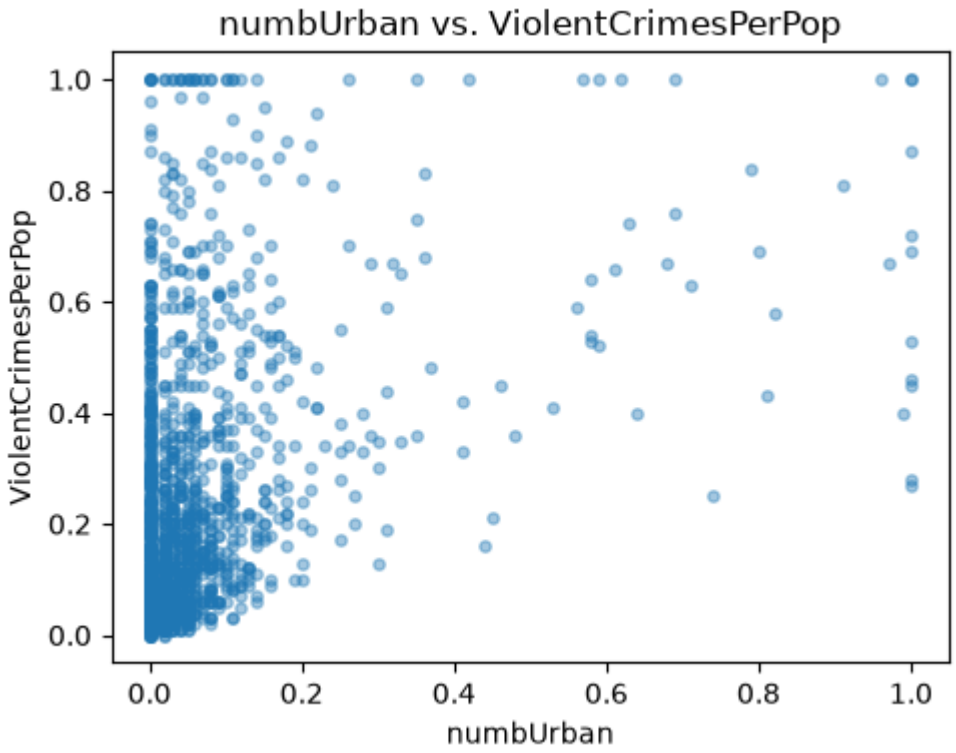

NumImmig vs. ViolentCrimesPerPop
ViolentCrimesPerPop
NumImmig
0.0
0.2
0.4
0.6
0.8
1.0

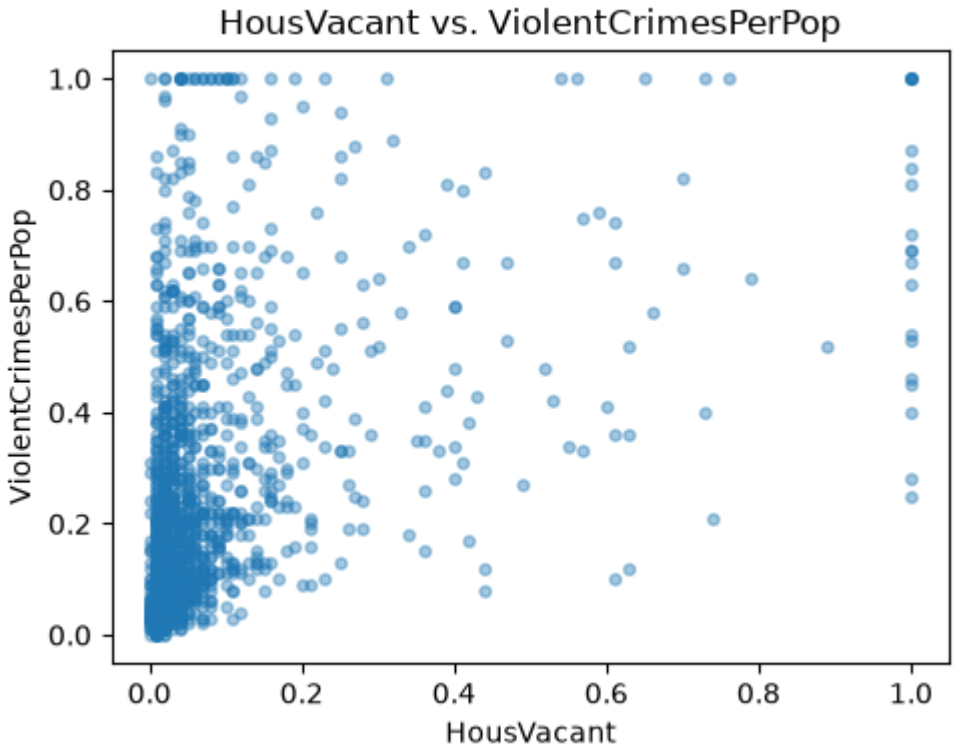

LemasPctOfficDrugUn vs. ViolentCrimesPerPop
ViolentCrimesPerPop
LemasPctOfficDrugUn
0.0
0.2
0.4
0.6
0.8
1.0


NumUnderPov vs. ViolentCrimesPerPop
ViolentCrimesPerPop
NumUnderPov
0.0
0.2
0.4
0.6
0.8
1.0


LemasSwornFT vs. ViolentCrimesPerPop
ViolentCrimesPerPop
LemasSwornFT
0.0
0.2
0.4
0.6
0.8
1.0


population vs. ViolentCrimesPerPop
ViolentCrimesPerPop
population
0.0
0.2
0.4
0.6
0.8
1.0



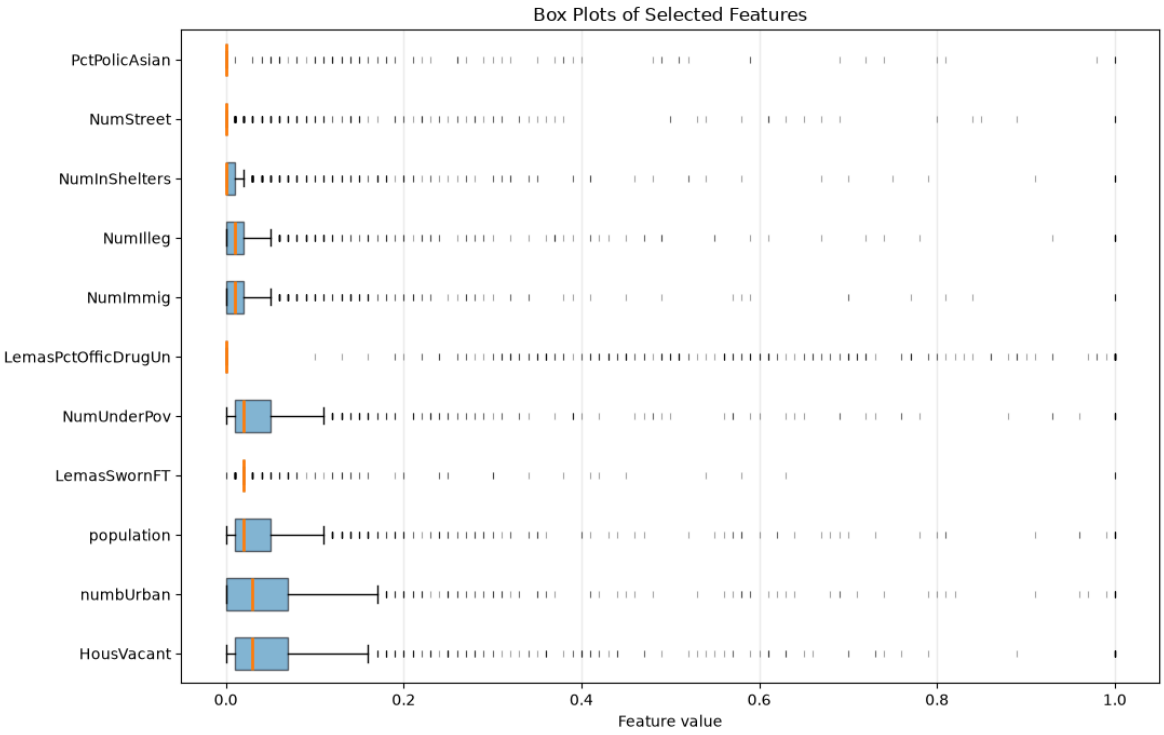

In [ ]:
```
data = [train_fill[name].to_numpy() for name in top]
plt.figure(figsize=(11, 7))
plt.boxplot(
    data,
    orientation="horizontal",
    tick_labels=top,
    showfliers=True,
    flierprops={
        "marker": "|",
        "markersize": 5,
        "markeredgewidth": 0.8,
        "alpha": 0.35,
        "linestyle": "none"
```

```
    },
    patch_artist=True,
    widths=0.55,
    boxprops={"alpha": 0.55},
    medianprops={"linewidth": 2}
)

plt.gca().invert_yaxis()
plt.gca().set_axisbelow(True)

plt.xlabel("Feature value")
plt.title("Box Plots of Selected Features")
plt.yticks(fontsize=10)
plt.grid(axis="x", alpha=0.25)

plt.tight_layout()
plt.show()
# plotting assist by GPT
```




No. The scatter plots may show relationships between the features and the target, but cannot tell whether these relationships are statistically significant from the plots alone

Still need an appropriate statistical test to draw that conclusion.

## (f) Fit a linear model

In [29]:
```
ols = LinearRegression()
ols.fit(train_fill, y_train)
pred = ols.predict(test_fill)
mse_ols = mean_squared_error(y_test, pred)

print(f"Test MSE: {mse_ols:.8f}")
```

```
Test MSE: 0.01796622
```

## (g) Fit a ridge regression model

In [30]:
```
method = imputer.strategy
folds = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
pipe = Pipeline([
    ("fill", SimpleImputer(strategy=method)),
    ("scale", StandardScaler()),
    ("model", Ridge())
])

params = {"model__alpha": np.logspace(-4, 4, 81)}
ridge = GridSearchCV(
    pipe,
    param_grid=params,
    scoring="neg_mean_squared_error",
    cv=folds,
    n_jobs=-1
)
ridge.fit(X_train, y_train)
pred = ridge.predict(X_test)
mse_ridge = mean_squared_error(y_test, pred)

print("Best alpha:", ridge.best_params_["model__alpha"])
print(f"CV MSE: {-ridge.best_score_:.8f}")
print(f"Test MSE: {mse_ridge:.8f}")
```

```
Best alpha: 125.89254117941688
CV MSE: 0.01973765
Test MSE: 0.01737249
```

slightly lower than the test MSE of ordinary linear regression

## (h) Fit a LASSO model

In [31]:
```
# without additional standardization
pipe = Pipeline([
    ("fill", SimpleImputer(strategy=method)),
    ("model", Lasso(max_iter=100000, tol=1e-6))
])
params = {"model__alpha": np.logspace(-5, 0, 51)}
lasso = GridSearchCV(
    pipe,
    param_grid=params,
    scoring="neg_mean_squared_error",
    cv=folds,
    n_jobs=-1
)
lasso.fit(X_train, y_train)
pred = lasso.predict(X_test)
mse_lasso = mean_squared_error(y_test, pred)

print("Best alpha:", lasso.best_params_["model__alpha"])
print(f"CV MSE: {-lasso.best_score_:.8f}")
```

```python
print(f"Test MSE: {mse_lasso:.8f}")

model = lasso.best_estimator_.named_steps["model"]
coef = pd.Series(
    model.coef_,
    index=X_train.columns,
    name="Coefficient"
)
keep = coef[coef.abs() > 1e-8]
print("Number of selected variables:", len(keep))
with pd.option_context("display.max_rows", None):
    display(keep.to_frame())
```

```
Best alpha: 0.00012589254117941674
CV MSE: 0.01952050
Test MSE: 0.01761464
Number of selected variables: 74
```

| | Coefficient |
|---|---|
| **racepctblack** | 0.267088 |
| **racePctAsian** | -0.014234 |
| **agePct12t21** | 0.057859 |
| **agePct12t29** | -0.246633 |
| **pctUrban** | 0.040552 |
| **pctWWage** | -0.076896 |
| **pctWFarmSelf** | 0.025244 |
| **pctWInvInc** | -0.096621 |
| **pctWSocSec** | 0.047017 |
| **pctWPubAsst** | 0.038210 |
| **pctWRetire** | -0.106650 |
| **whitePerCap** | -0.053323 |
| **blackPerCap** | -0.021215 |
| **indianPerCap** | -0.030344 |
| **AsianPerCap** | 0.028403 |
| **OtherPerCap** | 0.022711 |
| **HispPerCap** | 0.021852 |
| **PctPopUnderPov** | -0.136990 |
| **PctLess9thGrade** | -0.016722 |
| **PctEmploy** | 0.128365 |
| **PctEmplManu** | -0.014235 |
| **PctOccupManu** | 0.009879 |
| **MalePctDivorce** | 0.112631 |
| **MalePctNevMarr** | 0.122968 |
| **FemalePctDiv** | -0.071628 |
| **PctKids2Par** | -0.198265 |
| **PctYoungKids2Par** | -0.040009 |
| **PctTeen2Par** | -0.008165 |
| **PctWorkMom** | -0.127187 |
| **NumIlleg** | -0.078681 |
| **PctIlleg** | 0.101712 |
| **NumImmig** | -0.082098 |
| **PctImmigRecent** | -0.000390 |
| **PctImmigRec10** | -0.003686 |

| | Coefficient |
|---|---|
| **PctRecImmig10** | 0.035905 |
| **PctNotSpeakEnglWell** | -0.067297 |
| **PctLargHouseOccup** | -0.052332 |
| **PersPerOccupHous** | 0.030277 |
| **PctPersOwnOccup** | -0.053356 |
| **PctPersDenseHous** | 0.208404 |
| **PctHousLess3BR** | 0.053533 |
| **MedNumBR** | 0.002167 |
| **HousVacant** | 0.122695 |
| **PctHousOccup** | -0.061706 |
| **PctVacantBoarded** | 0.072535 |
| **PctVacMore6Mos** | -0.058225 |
| **MedYrHousBuilt** | -0.014097 |
| **PctHousNoPhone** | 0.022981 |
| **PctWOFullPlumb** | -0.023253 |
| **OwnOccLowQuart** | -0.008767 |
| **RentLowQ** | -0.153981 |
| **MedRent** | 0.152729 |
| **MedRentPctHousInc** | 0.089469 |
| **MedOwnCostPctInc** | -0.011290 |
| **MedOwnCostPctIncNoMtg** | -0.076610 |
| **NumInShelters** | 0.056119 |
| **NumStreet** | 0.156529 |
| **PctForeignBorn** | 0.047488 |
| **PctBornSameState** | -0.001133 |
| **PctSameCity85** | 0.024363 |
| **LemasSwFTFieldPerPop** | 0.069733 |
| **PolicReqPerOffic** | 0.089700 |
| **RacialMatchCommPol** | -0.016946 |
| **PctPolicBlack** | -0.011462 |
| **PctPolicHisp** | 0.012507 |
| **PctPolicAsian** | 0.021940 |
| **NumKindsDrugsSeiz** | -0.028411 |
| **PolicAveOTWorked** | -0.009110 |

| | Coefficient |
|---|---|
| **LandArea** | 0.027369 |
| **PopDens** | 0.010494 |
| **PctUsePubTrans** | -0.041979 |
| **PolicCars** | 0.013245 |
| **LemasPctPolicOnPatr** | 0.013937 |
| **LemasGangUnitDeploy** | 0.036368 |

standardized

In [ ]:
```
pipe = Pipeline([
    ("fill", SimpleImputer(strategy=method)),
    ("scale", StandardScaler()),
    ("model", Lasso(max_iter=100000, tol=1e-6))
])
scaled = GridSearchCV(
    pipe,
    param_grid=params,
    scoring="neg_mean_squared_error",
    cv=folds,
    n_jobs=-1
)
scaled.fit(X_train, y_train)
pred = scaled.predict(X_test)
mse_scaled = mean_squared_error(y_test, pred)

print("Best alpha:", scaled.best_params_["model__alpha"])
print(f"CV MSE: {-scaled.best_score_:.8f}")
print(f"Test MSE: {mse_scaled:.8f}")

model = scaled.best_estimator_.named_steps["model"]
coef_std = pd.Series(
    model.coef_,
    index=X_train.columns,
    name="Coefficient"
)
keep_std = coef_std[coef_std.abs() > 1e-8]
print("Number of selected variables:", len(keep_std))
with pd.option_context("display.max_rows", None):
    display(keep_std.to_frame())
# standardized assist by gpt
```

```
Best alpha: 0.000630957344480193
CV MSE: 0.01966094
Test MSE: 0.01772175
Number of selected variables: 76
```

| | Coefficient |
|---|---|
| **racepctblack** | 0.068005 |
| **racePctAsian** | -0.001863 |
| **agePct12t21** | 0.012487 |
| **agePct12t29** | -0.043304 |
| **pctUrban** | 0.017131 |
| **pctWWage** | -0.014238 |
| **pctWFarmSelf** | 0.004848 |
| **pctWInvInc** | -0.022143 |
| **pctWSocSec** | 0.010405 |
| **pctWPubAsst** | 0.007915 |
| **pctWRetire** | -0.017829 |
| **whitePerCap** | -0.009643 |
| **blackPerCap** | -0.003905 |
| **indianPerCap** | -0.005255 |
| **AsianPerCap** | 0.005442 |
| **OtherPerCap** | 0.004072 |
| **HispPerCap** | 0.004351 |
| **PctPopUnderPov** | -0.029375 |
| **PctLess9thGrade** | -0.003491 |
| **PctEmploy** | 0.025121 |
| **PctEmplManu** | -0.001756 |
| **PctOccupManu** | 0.000796 |
| **MalePctDivorce** | 0.021307 |
| **MalePctNevMarr** | 0.024740 |
| **FemalePctDiv** | -0.014646 |
| **PctKids2Par** | -0.040691 |
| **PctYoungKids2Par** | -0.007760 |
| **PctTeen2Par** | -0.001961 |
| **PctWorkMom** | -0.022406 |
| **NumIlleg** | -0.010262 |
| **PctIlleg** | 0.023937 |
| **NumImmig** | -0.011299 |
| **PctImmigRecent** | -0.000015 |
| **PctImmigRec10** | -0.000660 |

| | Coefficient |
|---|---|
| **PctRecImmig10** | 0.008689 |
| **PctNotSpeakEnglWell** | -0.013186 |
| **PctLargHouseOccup** | -0.012075 |
| **PersPerOccupHous** | 0.009263 |
| **PersPerOwnOccHous** | -0.000378 |
| **PctPersOwnOccup** | -0.011470 |
| **PctPersDenseHous** | 0.043358 |
| **PctHousLess3BR** | 0.011284 |
| **MedNumBR** | 0.000736 |
| **HousVacant** | 0.017531 |
| **PctHousOccup** | -0.011884 |
| **PctVacantBoarded** | 0.015072 |
| **PctVacMore6Mos** | -0.011004 |
| **MedYrHousBuilt** | -0.003116 |
| **PctHousNoPhone** | 0.003597 |
| **PctWOFullPlumb** | -0.005134 |
| **OwnOccLowQuart** | -0.001175 |
| **RentLowQ** | -0.030878 |
| **MedRent** | 0.030542 |
| **MedRentPctHousInc** | 0.015276 |
| **MedOwnCostPctInc** | -0.003632 |
| **MedOwnCostPctIncNoMtg** | -0.014630 |
| **NumInShelters** | 0.008503 |
| **NumStreet** | 0.017465 |
| **PctForeignBorn** | 0.008167 |
| **PctSameCity85** | 0.003505 |
| **LemasSwFTFieldPerPop** | 0.007139 |
| **LemasTotalReq** | -0.005465 |
| **PolicReqPerOffic** | 0.009655 |
| **RacialMatchCommPol** | -0.003170 |
| **PctPolicBlack** | -0.003284 |
| **PctPolicHisp** | 0.003054 |
| **PctPolicAsian** | 0.002086 |
| **OfficAssgnDrugUnits** | -0.002675 |

| | Coefficient |
|---|---|
| **NumKindsDrugsSeiz** | -0.003474 |
| **PolicAveOTWorked** | -0.001939 |
| **LandArea** | 0.005227 |
| **PopDens** | 0.003541 |
| **PctUsePubTrans** | -0.009777 |
| **PolicCars** | 0.007770 |
| **LemasPctPolicOnPatr** | 0.001766 |
| **LemasGangUnitDeploy** | 0.006294 |

In [33]:
```
table = pd.DataFrame({
    "Model": ["LASSO", "LASSO (standardized)"],
    "Alpha": [
        lasso.best_params_["model__alpha"],
        scaled.best_params_["model__alpha"]
    ],
    "Selected variables": [len(keep), len(keep_std)],
    "Test MSE": [mse_lasso, mse_scaled]
})

display(table.round(8))
```

| | Model | Alpha | Selected variables | Test MSE |
|---|---|---|---|---|
| **0** | LASSO | 0.000126 | 74 | 0.017615 |
| **1** | LASSO (standardized) | 0.000631 | 76 | 0.017722 |

Without standardization, LASSO selects 74 variables and gives a test MSE of 0.01761464. After standardization, it selects 76 variables and gives a test MSE of 0.01772175.

model without standardization performs slightly better on the test set, but the difference is small.

## (i) Fit a PCR model

In [ ]:
```
pipe = Pipeline([
    ("fill", SimpleImputer(strategy=method)),
    ("scale", StandardScaler()),
    ("pca", PCA(svd_solver="full")),
    ("model", LinearRegression())
])
params = {"pca__n_components": list(range(1, X_train.shape[1]+1))}
pcr = GridSearchCV(
    pipe,
    param_grid=params,
    scoring="neg_mean_squared_error",
    cv=folds,
    n_jobs=-1
```

```
)
pcr.fit(X_train, y_train)

M = pcr.best_params_["pca__n_components"]
pred = pcr.predict(X_test)
mse_pcr = mean_squared_error(y_test, pred)

print("Best number of components:", M)
print(f"CV MSE: {-pcr.best_score_:.8f}")
print(f"Test MSE: {mse_pcr:.8f}")
```

```
Best number of components: 60
CV MSE: 0.02020950
Test MSE: 0.01834953
```

use 5-fold cross-validation to choose the number of principal components

The lowest CV MSE is obtained with 60 components

fit the PCR model on the training set using these components and obtain a test MSE of 0.01834953.

## (j) Fit a boosting tree

In [ ]:

```
pipe = Pipeline([
    ("fill", SimpleImputer(strategy=method)),
    ("model", XGBRegressor(
        objective="reg:squarederror",
        booster="gbtree",
        tree_method="hist",
        n_estimators=300,
        max_depth=3,
        learning_rate=0.05,
        reg_lambda=1.0,
        random_state=42,
        n_jobs=1
    ))
])

alphas = np.concatenate(([0.0], np.logspace(-4, 2, 25)))
params = {"model__reg_alpha": alphas}
boost = GridSearchCV(
    pipe,
    param_grid=params,
    scoring="neg_mean_squared_error",
    cv=folds,
    n_jobs=-1,
    verbose=2,
    pre_dispatch=2,
    error_score="raise"
)
boost.fit(X_train, y_train)
alpha = boost.best_params_["model__reg_alpha"]
pred = boost.predict(X_test)
mse_boost = mean_squared_error(y_test, pred)
# GPT assist with troubleshoot the XGBoost grid search by suggesting fewer paral
print(f"Best alpha: {alpha:.6g}")
```

```
print(f"CV MSE: {-boost.best_score_:.8f}")
print(f"Test MSE: {mse_boost:.8f}")
```

```
Fitting 5 folds for each of 26 candidates, totalling 130 fits
[CV] END ..............................model__reg_alpha=0.0; total time=   1.6s
[CV] END ..............................model__reg_alpha=0.0; total time=   1.6s
[CV] END ..............................model__reg_alpha=0.0; total time=   1.0s
[CV] END ..............................model__reg_alpha=0.0; total time=   1.0s
[CV] END ..............................model__reg_alpha=0.0; total time=   1.0s
[CV] END ............................model__reg_alpha=0.0001; total time=   1.0s
[CV] END ............................model__reg_alpha=0.0001; total time=   1.2s
[CV] END ............................model__reg_alpha=0.0001; total time=   1.2s
[CV] END ............................model__reg_alpha=0.0001; total time=   1.2s
[CV] END ............................model__reg_alpha=0.0001; total time=   1.2s
[CV] END ............model__reg_alpha=0.00017782794100389227; total time=   2.0s
[CV] END ............model__reg_alpha=0.00017782794100389227; total time=   2.1s
[CV] END ............model__reg_alpha=0.00017782794100389227; total time=   1.5s
[CV] END ............model__reg_alpha=0.00017782794100389227; total time=   1.4s
[CV] END ............model__reg_alpha=0.00017782794100389227; total time=   4.1s
[CV] END ............model__reg_alpha=0.00031622776601683794; total time=   4.1s
[CV] END ............model__reg_alpha=0.00031622776601683794; total time=   0.8s
[CV] END ............model__reg_alpha=0.00031622776601683794; total time=   0.8s
[CV] END ............model__reg_alpha=0.00031622776601683794; total time=   0.8s
[CV] END ............model__reg_alpha=0.00031622776601683794; total time=   0.8s
[CV] END ............model__reg_alpha=0.00056234132519034913; total time=   0.8s
[CV] END ............model__reg_alpha=0.00056234132519034913; total time=   0.8s
[CV] END ............model__reg_alpha=0.00056234132519034913; total time=   0.7s
[CV] END ............model__reg_alpha=0.00056234132519034913; total time=   0.8s
[CV] END ............model__reg_alpha=0.00056234132519034913; total time=   0.8s
[CV] END ............................model__reg_alpha=0.001; total time=   0.8s
[CV] END ............................model__reg_alpha=0.001; total time=   0.6s
[CV] END ............................model__reg_alpha=0.001; total time=   0.5s
[CV] END ............................model__reg_alpha=0.001; total time=   0.6s
[CV] END ............................model__reg_alpha=0.001; total time=   0.7s
[CV] END ............model__reg_alpha=0.0017782794100389228; total time=   0.9s
[CV] END ............model__reg_alpha=0.0017782794100389228; total time=   0.8s
[CV] END ............model__reg_alpha=0.0017782794100389228; total time=   0.7s
[CV] END ............model__reg_alpha=0.0017782794100389228; total time=   0.7s
[CV] END ............model__reg_alpha=0.0017782794100389228; total time=   0.6s
[CV] END ............model__reg_alpha=0.0031622776601683794; total time=   0.6s
[CV] END ............model__reg_alpha=0.0031622776601683794; total time=   0.5s
[CV] END ............model__reg_alpha=0.0031622776601683794; total time=   0.6s
[CV] END ............model__reg_alpha=0.0031622776601683794; total time=   0.6s
[CV] END ............model__reg_alpha=0.0031622776601683794; total time=   0.5s
[CV] END ............model__reg_alpha=0.0056234132519034913; total time=   0.7s
[CV] END ............model__reg_alpha=0.0056234132519034913; total time=   0.7s
[CV] END ............model__reg_alpha=0.0056234132519034913; total time=   0.7s
[CV] END ............model__reg_alpha=0.0056234132519034913; total time=   0.7s
[CV] END ............................model__reg_alpha=0.01; total time=   0.6s
[CV] END ............model__reg_alpha=0.0056234132519034913; total time=   0.6s
[CV] END ............................model__reg_alpha=0.01; total time=   0.6s
[CV] END ............................model__reg_alpha=0.01; total time=   0.7s
[CV] END ............................model__reg_alpha=0.01; total time=   0.7s
[CV] END ............................model__reg_alpha=0.01; total time=   0.7s
[CV] END ............model__reg_alpha=0.017782794100389228; total time=   0.7s
[CV] END ............model__reg_alpha=0.017782794100389228; total time=   0.7s
[CV] END ............model__reg_alpha=0.017782794100389228; total time=   0.7s
[CV] END ............model__reg_alpha=0.017782794100389228; total time=   0.7s
[CV] END ............model__reg_alpha=0.017782794100389228; total time=   0.7s
[CV] END ............model__reg_alpha=0.031622776601683794; total time=   0.7s
[CV] END ............model__reg_alpha=0.031622776601683794; total time=   0.7s
[CV] END ............model__reg_alpha=0.031622776601683794; total time=   0.8s
[CV] END ............model__reg_alpha=0.031622776601683794; total time=   0.8s
```

```
[CV] END ..............model__reg_alpha=0.03162277660168379; total time=   0.8s
[CV] END ..............model__reg_alpha=0.05623413251903491; total time=   0.9s
[CV] END ..............model__reg_alpha=0.05623413251903491; total time=   0.9s
[CV] END ..............model__reg_alpha=0.05623413251903491; total time=   0.8s
[CV] END ..............model__reg_alpha=0.05623413251903491; total time=   0.8s
[CV] END ..............model__reg_alpha=0.05623413251903491; total time=   0.8s
[CV] END ............................model__reg_alpha=0.1; total time=   0.8s
[CV] END ............................model__reg_alpha=0.1; total time=   0.8s
[CV] END ............................model__reg_alpha=0.1; total time=   0.9s
[CV] END ............................model__reg_alpha=0.1; total time=   0.9s
[CV] END ............................model__reg_alpha=0.1; total time=   0.9s
[CV] END ..............model__reg_alpha=0.1778279410038923; total time=   0.9s
[CV] END ..............model__reg_alpha=0.1778279410038923; total time=   0.8s
[CV] END ..............model__reg_alpha=0.1778279410038923; total time=   0.8s
[CV] END ..............model__reg_alpha=0.1778279410038923; total time=   0.8s
[CV] END ..............model__reg_alpha=0.1778279410038923; total time=   0.8s
[CV] END ..............model__reg_alpha=0.31622776601683794; total time=   0.8s
[CV] END ..............model__reg_alpha=0.31622776601683794; total time=   0.7s
[CV] END ..............model__reg_alpha=0.31622776601683794; total time=   0.8s
[CV] END ..............model__reg_alpha=0.31622776601683794; total time=   0.9s
[CV] END ..............model__reg_alpha=0.31622776601683794; total time=   0.8s
[CV] END ..............model__reg_alpha=0.5623413251903491; total time=   0.8s
[CV] END ..............model__reg_alpha=0.5623413251903491; total time=   0.8s
[CV] END ..............model__reg_alpha=0.5623413251903491; total time=   0.9s
[CV] END ..............model__reg_alpha=0.5623413251903491; total time=   0.9s
[CV] END ..............model__reg_alpha=0.5623413251903491; total time=   0.8s
[CV] END ............................model__reg_alpha=1.0; total time=   0.8s
[CV] END ............................model__reg_alpha=1.0; total time=   0.6s
[CV] END ............................model__reg_alpha=1.0; total time=   0.6s
[CV] END ............................model__reg_alpha=1.0; total time=   0.6s
[CV] END ............................model__reg_alpha=1.0; total time=   0.6s
[CV] END ..............model__reg_alpha=1.7782794100389228; total time=   0.6s
[CV] END ..............model__reg_alpha=1.7782794100389228; total time=   0.6s
[CV] END ..............model__reg_alpha=1.7782794100389228; total time=   0.6s
[CV] END ..............model__reg_alpha=1.7782794100389228; total time=   0.6s
[CV] END ..............model__reg_alpha=3.1622776601683795; total time=   0.4s
[CV] END ..............model__reg_alpha=1.7782794100389228; total time=   0.5s
[CV] END ..............model__reg_alpha=3.1622776601683795; total time=   0.4s
[CV] END ..............model__reg_alpha=3.1622776601683795; total time=   0.4s
[CV] END ..............model__reg_alpha=3.1622776601683795; total time=   0.4s
[CV] END ..............model__reg_alpha=3.1622776601683795; total time=   0.4s
[CV] END ..............model__reg_alpha=5.623413251903491; total time=   0.4s
[CV] END ..............model__reg_alpha=5.623413251903491; total time=   0.3s
[CV] END ..............model__reg_alpha=5.623413251903491; total time=   0.3s
[CV] END ..............model__reg_alpha=5.623413251903491; total time=   0.3s
[CV] END ..............model__reg_alpha=5.623413251903491; total time=   0.3s
[CV] END ............................model__reg_alpha=10.0; total time=   0.3s
[CV] END ............................model__reg_alpha=10.0; total time=   0.3s
[CV] END ............................model__reg_alpha=10.0; total time=   0.4s
[CV] END ............................model__reg_alpha=10.0; total time=   0.4s
[CV] END ............................model__reg_alpha=10.0; total time=   0.3s
[CV] END ..............model__reg_alpha=17.78279410038923; total time=   0.4s
[CV] END ..............model__reg_alpha=17.78279410038923; total time=   0.4s
[CV] END ..............model__reg_alpha=17.78279410038923; total time=   0.4s
[CV] END ..............model__reg_alpha=17.78279410038923; total time=   0.4s
[CV] END ..............model__reg_alpha=17.78279410038923; total time=   0.4s
[CV] END ..............model__reg_alpha=31.622776601683793; total time=   0.3s
[CV] END ..............model__reg_alpha=31.622776601683793; total time=   0.3s
[CV] END ..............model__reg_alpha=31.622776601683793; total time=   0.3s
[CV] END ..............model__reg_alpha=31.622776601683793; total time=   0.3s
```

```
[CV] END ..............model__reg_alpha=31.622776601683793; total time=   0.3s
[CV] END ..............model__reg_alpha=56.23413251903491; total time=   0.3s
[CV] END ..............model__reg_alpha=56.23413251903491; total time=   0.3s
[CV] END ..............model__reg_alpha=56.23413251903491; total time=   0.3s
[CV] END ..............model__reg_alpha=56.23413251903491; total time=   0.3s
[CV] END ............................model__reg_alpha=100.0; total time=   0.2s
[CV] END ..............model__reg_alpha=56.23413251903491; total time=   0.3s
[CV] END ............................model__reg_alpha=100.0; total time=   0.2s
[CV] END ............................model__reg_alpha=100.0; total time=   0.2s
[CV] END ............................model__reg_alpha=100.0; total time=   0.2s
[CV] END ............................model__reg_alpha=100.0; total time=   0.2s
Best alpha: 0.1
CV MSE: 0.01990737
Test MSE: 0.01642190
```

use XGBoost and choose the L1 regularization parameter by 5-fold cross-validation

with other parameters fixed, the selected alpha is 0.1

mean CV MSE is 0.01990737, test MSE is 0.01642190

XGBoost has the lowest test MSE among these models

# honor code

All codes completed with AI assistance have been identified in the comments